

Web 2.0, Interactive and Dynamic Websites



The evolution of the "primordial" Web

What is Web 2.0?...

Web 2.0 was not born as a new technology, but rather, as a set of already existing technologies

The W3C was not directly involved...

Term used for the first time by Tim O'Reilly at a conference in 2004

Founder and CEO of O'Reilly Media, Inc.

[<https://www.oreilly.com/pub/a/web2/archive/what-is-web-20.html>]

Web 2.0 is a “new way of using the Web”

Users, who were passive surfers in the "Web 1.0", in the Web 2.0 become active players who provide their own content

The evolution of the "primordial" Web

... what is Web 2.0?

Web 2.0 sites extensively use those technologies that allow the implementation of interactivity and dynamicity

Both client-side and (above all) server-side technologies: HTML and CSS are not enough

The term *Web 2.0* is commonly associated with web applications that facilitate interactive information sharing, interoperability, user-centered design, and collaboration on the World Wide Web

Examples of Web 2.0 include web-based communities, hosted services, web applications, social-networking websites, video-sharing sites, wikis, blogs, mashups, and folksonomies

The protagonists
Web 2.0



Online diaries: *blogs*

From *Web log*

"Online journals" by single users or organizations

Frequent updates (usually)

E.g., daily – “Posts” (chronologically ordered) can include text, images, and multimedia elements

Readers can generally provide their own replies to the author’s posts

Server-side technologies are needed – there are also online services that do (almost) everything by themselves, creating all the necessary “technological infrastructure” (interface, database, etc.)

Social networks

Facebook is (was?) the most famous, but not the only one

There are many social networks...

[https://en.wikipedia.org/wiki/List_of_social_networking_websites]

- Twitter/X
- LinkedIn
- TikTok
- Xing
- MySpace
- ...



It is also possible to build personalized social networks

E.g., NING [<https://www.ning.com/>]

Wiki

WikiWikiWeb

- First wiki ever, created by Ward Cunningham in 1994
[<http://c2.com/cgi/wiki?WikiWikiWeb>]
- Word "wiki" inspired by the Honolulu airport's bus, called "Wiki wiki" (= fast)



Wikipedia

The most famous example

Several open source “platforms” exist for the creation of wiki services

E.g., MediaWiki [<https://www.mediawiki.org/wiki/MediaWiki>]



Some Web 2.0 services

Content sharing

Video sharing

E.g., YouTube, Vimeo



Photo sharing

E.g., Instagram, Flickr

Presentation sharing

E.g., Slideshare

...

RSS Feeds

Acronym of Really Simple Syndication

An XML-based format for “automatic” delivery of content
(originally used by journalists)



How do they work?

- It is necessary to subscribe to feeds on websites that offer them; then, a *reader* periodically checks whether there are any “changes” in those websites (i.e., there are new feeds). Several free feed readers exist. At least initially, RSS feeds were a sign of “regard” towards the user
- Almost a synonym of “active” website

From *products* to *services*

Typical example: *Netscape* vs. *Google*

- Netscape was born as a company that produced software, i.e., a *product* (the web browser, that needed to be installed, updated, etc.)
- Google was born as a web *service*



Online applications as a natural evolution of offline applications

- Users as “co-developers” (quick feedback about product usage)
- Platform independence

Cloud computing...

Everything is on the Internet

Everything is “out there”, delocalized, distributed... in the cloud!

One of the first (real) examples of cloud computing:

Google Chrome OS (2009)

Google Chrome OS is an open source, lightweight operating system that will initially be targeted at netbooks. [...] We're designing the OS to be fast and lightweight, to start up and get you onto the web in a few seconds. The user interface is minimal to stay out of your way, and most of the user experience takes place on the web.

[<https://googleblog.blogspot.com/2009/07/introducing-google-chrome-os.html>]



Why the Web 2.0?

... cloud computing...

Use of remote servers to store and/or process data

Main features

- Need for a constant internet connection
- Use "on-demand"
- Flexibility
- Pay per use



Why the Web 2.0?

... cloud computing...

SaaS = Software as a Service

- Applications similar to desktop programs that are executed within the web browser (RIA = Rich Internet Applications)
- Example: Google *Workspace* (Gmail, Docs, Drive, Calendar, ...)

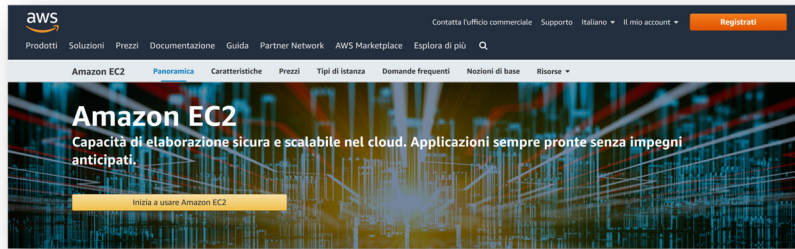
PaaS = Platform as a Service

- Host services to develop and execute applications in the cloud
- Examples: Amazon *Elastic Compute Cloud* (EC2) and Microsoft *Azure*



Why the Web 2.0?

... cloud computing...



Why the Web 2.0?

... cloud computing

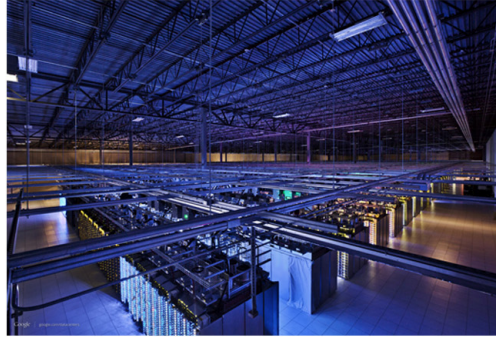
Potential problems

- Possible conflicts with international regulations
- Data ownership
- Guarantee of service
- Security
- ...



Data centers

Example: Google



Rich Internet Applications

Rich Internet Applications (RIA) are “web applications” with features similar to those of desktop programs

They work within a web browser – in practice, they are web pages that partially or totally behave as ordinary applications

RIAs are usually very interactive

RIAs are especially suitable for Web 2.0

They are its engine

Front end vs back end

Front end

The "visible part" (for example, of a website) with which users interact – typically implemented with client-side technologies

Back end

The "hidden part" (for example, of a website), which performs data processing and client-server interaction – typically implemented with server-side technologies



Interactive websites

HTML alone does not allow for the definition of user interactions

Examples of interactions:

- Image that changes when the mouse cursor is over it
- Page content that changes when a button is pressed
- Pop-up window that displays an error message

To create interactions, it is usually necessary to write programming code

With CSS3, however, some simple forms of interaction become possible

Client-side vs. Server-side Interactions

In *client-side interaction*, the interactive behavior of the page is defined by code executed on the user's device

E.g., *JavaScript*

In *server-side interaction*, the interactive behavior of the page is defined by code executed on the server

E.g., *CGI programs and application servers*

Main (basic) client-side technologies

- JavaScript
- In the past:
 - Adobe Flash
 - Java applets
 - ActiveX controls (Windows)



DHTML

DHTML = *Dynamic* HTML

It is not a new HTML, but the joint use of HTML, CSS, and JavaScript

DHTML allows for the creation of interactive (and sometimes dynamic) behaviors

Through JavaScript, it is possible to modify styles “on the fly”

E.g., pressing a button causes the page's background color to change; moving the mouse pointer over an image causes the image to be substituted with another one; etc.

Dynamic websites...

A *dynamic* website is a website whose content (or part of it) is **generated "on demand"**, i.e., only when necessary

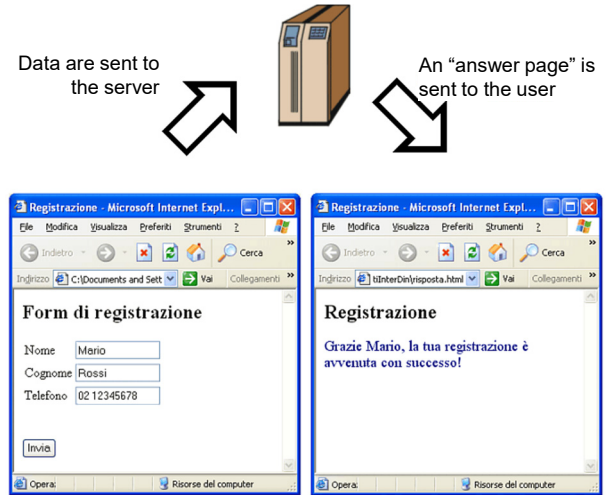
- E.g., websites whose content is driven by a database: data is not stored in an HTML file, but is generated “on the fly” when pages are requested
- The dynamic generation of content is usually performed by **CGI programs** or **application servers** (which stay on the server)

But a page can also be made dynamic by client-side technologies

... dynamic websites...

Example: form input is processed on the server...

...the user may then receive an “answer page”
whose content is not statically defined, but
depends on the data provided to the form



... dynamic websites

Important: **interactive** vs. **dynamic** websites

- A website can be interactive without being dynamic
E.g., a page containing a rollover effect
- A website can be dynamic without being interactive
E.g., an online newspaper that displays news read from a database

CGI programs...

- Common Gateway Interface
- Can be written in (almost) any programming language
- Typical use: form data processing (e.g., database interaction) and dynamic content generation (sent to the client and displayed by the browser)

E.g., form whose data will be sent to the CGI program `process.tcl`:

```
<form name="f" method="get" action="/cgi-bin/process.tcl">
```

... CGI programs...

GET and POST are two different ways to send data to the server

Example of a request to a *CGI script*:

```
https://myhost.com/mypath/myscript.cgi?var1=val1&var2=val2
```

The string `var1=val1&var2=val2` is the input to the script

GET method

The string is contained in the “environment variable” `QUERY_STRING`

The sent data is visible in the generated URL

POST method

The string is obtained from the “standard input”

... CGI programs

Example of CGI program (written in Python):

```
<form method="get" action="/cgi-bin/get.py">  
  Name: <input type="text" name="n"><br>  
  Surname: <input type="text" name="s">  
  <input type="submit" value="Send">  
</form>
```

Form that sends the data to *get.py*

get.py

```
#!/usr/bin/python  
import cgi, cgitb  
form = cgi.FieldStorage()  
n = form.getvalue('n')  
s = form.getvalue('s')  
print("Content-type:text/html\r\n\r\n")  
print("<html>")  
print("  <head>")  
print("    <title>Test: CGI in Python</title>")  
print("  </head>")  
print("  <body>")  
print("    <h1>Ciao " + n + s + "!</h1>")  
print("  </body>")  
print("</html>")
```

Application servers

- Software installed on a server (usually, but not necessarily, the computer hosting the web server) that can interpret and execute programming code written in a specific language; PHP is probably the most widespread application server
- Purposes analogous to those of CGI programs
- Unlike CGI programs, the programming code can be integrated into HTML

Faster interactions: AJAX



Acronym of Aynchronous JavaScript And XML

It is not a new technology, but the asynchronous use of JavaScript: a client can request and get server-side data in a faster way

Asynchronous requests to the server

The `XMLHttpRequest` object allows requests to be sent to the server without waiting for its answer (thus, independently of the browser) – E.g., *Google Maps*

Today, AJAX often uses *JSON* instead of XML for data exchange

Web 2.0 and SEO

SEO = Search Engine Optimization

Strategies to improve website ranking



A proper “SEO policy” may be vital for a company to survive

If Google doesn’t find you, you don’t exist!

With Web 2.0, SEO strategies must consider new factors

Forums, social networks, blogs, etc. (**strongly**) concur to influence search engines’ results